

Lecture 6: Error Detection/Correction - CRC

EC 441 - Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Apply the single-bit error correction theorem to determine minimum codeword length
2. Quantitatively compare error detection, correction, and repetition coding
3. Compute block error probabilities using the binomial distribution
4. Describe CRC codes using polynomial notation: $G(x)$, $T(x)$, $M(x)$, $R(x)$
5. Perform CRC encoding and decoding using polynomial (modulo-2) division
6. Identify standard CRC polynomials and their error detection capabilities

— Session 1 (40 minutes) —

1 Quick Review: Block Codes and $d_{\min}$ (5 min)

1.1 Recap from Lecture 5

Block code (n, k) : k data bits encoded into n -bit codewords.

- **Code rate:** $R_c = k/n$
- **Hamming distance:** Number of bit positions where two words differ
- **Minimum distance** $d_{\min}$: Smallest Hamming distance between any pair of distinct codewords

The $d_{\min}$ theorem:

$$e_c + e_d \leq d_{\min} - 1, \quad e_c \leq e_d \tag{1}$$

Quick reference:

- To **detect** d errors: need $d_{\min} \geq d + 1$
- To **correct** d errors: need $d_{\min} \geq 2d + 1$

2 Single-Bit Error Correction Theorem (10 min)

2.1 Theorem

Single-bit error correction requires the codeword length n to satisfy:

$$\boxed{n + 1 \leq 2^{n-k}} \quad (2)$$

If n is less than the minimum value satisfying this inequality, the code **cannot** perform single-bit error correction.

2.2 Proof

For single-bit error correction ($e_c = 1$), each codeword needs $n + 1$ words reserved for itself:

- The codeword itself (0 errors)
- The n words that differ in exactly one bit position

Since there are 2^k codewords, we need at least $2^k(n + 1)$ distinct words. But there are only 2^n possible words total. Therefore:

$$2^k(n + 1) \leq 2^n \quad (3)$$

Dividing both sides by 2^k :

$$n + 1 \leq 2^{n-k} \quad (4)$$

2.3 Discussion

- This is a **necessary** condition (not sufficient) — satisfying the inequality does not guarantee a code exists
- As n increases, the right side (2^{n-k}) grows exponentially, so the inequality eventually holds
- The condition puts a **limit on the maximum frame length** for a given number of check bits

Minimum Check Bits for 1000-bit Frames

For $k = 1000$ data bits, find the minimum number of check bits for single-bit error correction. Solve $n + 1 \leq 2^{n-1000}$.

Testing:

- $n = 1009$: $1010 \leq 2^9 = 512$ — **No**
- $n = 1010$: $1011 \leq 2^{10} = 1024$ — **Yes**

Therefore we need $n - k = 10$ check bits minimum for single-bit error correction of 1000-bit frames.

3 BER Comparison: Detection vs. Correction vs. Repetition (15 min)

3.1 Problem Setup

Compare utilization U and residual probability of bit error P_e for three methods:

1. Single-bit error correction
2. Single-bit error detection (with retransmission)
3. Rate 1/3 repetition coding with error correction

Assumptions:

- Frame size: $n = 1000$ bits
- Channel BER: $p = 10^{-6}$
- Use the most efficient codes possible
- Retransmit if there is a detected error

3.2 Block Error Probability

Errors in each bit are independent, so the number of errors in a block is **binomial** with $N = 1000$, $p = 10^{-6}$:

$$f(w) = \binom{N}{w} p^w (1-p)^{N-w} \quad (5)$$

Computing key values:

$$f(0) = (1-p)^{1000} \approx 0.999 \quad (\text{probability of zero errors}) \quad (6)$$

$$f(1) = \binom{1000}{1} p (1-p)^{999} \approx 0.001 \quad (\text{exactly 1 error}) \quad (7)$$

$$f(2) \approx 10^{-6.3} \quad (\text{exactly 2 errors}) \quad (8)$$

$$f(3) \approx 10^{-9.8} \quad (\text{exactly 3 errors}) \quad (9)$$

3.3 Method 1: Single-Bit Error Correction

From the single-bit error correction theorem: $(1000 + 1) \leq 2^{n-k}$, so $n - k = 10$ check bits.

Utilization: No retransmissions needed, so:

$$U = \frac{k}{n} = \frac{990}{1000} = 0.990 \quad (10)$$

Residual BER: Errors occur when 2 or more bits are wrong (since we can only correct 1):

$$P_e = \frac{1}{N} \sum_{q=2}^N q \cdot f(q) \approx \frac{2 \times 10^{-6.3} + 3 \times 10^{-9.8}}{1000} \approx 10^{-9} \quad (11)$$

3.4 Method 2: Single-Bit Error Detection

Use $n - k = 1$ check bit (parity bit code).

Utilization: The number of transmissions M is geometric with success probability $1 - f(1) - f(2) - \dots \approx 1 - 10^{-3}$, so $E[M] = 1/(1 - 10^{-3})$:

$$U = \frac{k}{n \cdot E[M]} = \frac{999}{1000 \times \frac{1}{0.999}} = \frac{998}{1000} = 0.998 \quad (12)$$

Residual BER: Of accepted frames, some have 2 or more undetected errors (even number of errors go undetected by parity). Same order as error correction:

$$P_e \approx 10^{-9} \quad (13)$$

3.5 Method 3: Rate 1/3 Repetition Coding

Approximate: $n = 999$, $k = 333$. Each bit is transmitted 3 times and decoded by majority vote.

Utilization:

$$U = R_c = \frac{1}{3} = 0.333 \quad (14)$$

Residual BER: A bit error occurs when 2 of 3 or 3 of 3 copies are flipped:

$$P_e = \binom{3}{2} p^2(1 - p) + p^3 = 3 \times 10^{-12} + 10^{-18} \approx 3 \times 10^{-12} \quad (15)$$

3.6 Results Summary

Method	Utilization U	Residual P_e
Single-bit error detection (parity)	0.998	$\sim 10^{-9}$
Single-bit error correction	0.990	$\sim 10^{-9}$
Rate 1/3 repetition coding	0.333	$\sim 3 \times 10^{-12}$

Table 1: Comparison of error control methods for $n = 1000$, $\text{BER} = 10^{-6}$

3.7 Key Insights

- The **rate 1/3 code is very inefficient** in throughput ($U = 0.333$)
- **Error detection is approximately 5 times more efficient** than error correction in utilization for this scenario (typical result)
- For networking (where retransmission is possible), **error detection wins**
- Error correction avoids retransmission but uses more overhead bits
- Repetition coding achieves better P_e but at enormous throughput cost

4 Design Example 3 (10 min)

Design Example 3: Codeword Analysis

An error control code uses the following set of codewords:

$$\mathcal{C} = \{00000, 11100, 11011, 00111\}$$

- (a) What is the rate of the code?
- (b) What is the minimum distance of the code?
- (c) If used for error detection only, how many bit errors are guaranteed to be detected?
- (d) If used for error correction only, how many bit errors are guaranteed to be corrected?
- (e) What should you do if you receive 01010 and are using error correction?

Solution

(a) There are 4 codewords, so $2^k = 4 \implies k = 2$. The codeword length is $n = 5$. The rate is:

$$R_c = \frac{k}{n} = \frac{2}{5} = 0.4$$

(b) Compute all pairwise Hamming distances:

Pair	Distance
dist(00000, 11100)	3
dist(00000, 11011)	4
dist(00000, 00111)	3
dist(11100, 11011)	3
dist(11100, 00111)	4
dist(11011, 00111)	3

Therefore $d_{\min} = 3$.

(c) Error detection only ($e_c = 0$): can detect $e_d = d_{\min} - 1 = 2$ bit errors.

(d) Error correction only ($e_c = e_d$): can correct $e_c = \lfloor (d_{\min} - 1)/2 \rfloor = 1$ bit error.

Note: the necessary condition is satisfied: $n + 1 = 6 \leq 2^{n-k} = 2^3 = 8$.

(e) The word 01010 is:

- Distance 2 from 00000
- Distance 3 from 11100
- Distance 2 from 11011
- Distance 3 from 00111

Since 01010 is equidistant from two codewords (00000 and 11011), the receiver **cannot reliably correct** this error. This is a 2-bit error pattern, which exceeds the single-bit error correction capability of the code. Options:

- Request retransmission (if possible)
- Guess either 00000 or 11011 (50% chance of being correct)

— Session 2 (40 minutes) —

5 CRC Codes: Polynomial Representation (15 min)

5.1 Cyclic Redundancy Check (CRC)

CRC codes are the most widely used error detection codes in networking. They treat bit strings as polynomials over GF(2) (binary arithmetic with no carries).

Symbol	Meaning	Size
$M(x)$	Message (data) polynomial	k bits
$G(x)$	Generator polynomial	$r + 1$ bits
$R(x)$	Remainder (CRC checksum)	r bits
$T(x)$	Transmitted frame (codeword)	$n = r + k$ bits

Table 2: CRC polynomial notation

5.2 Polynomial Notation

5.3 Binary Polynomial Representation

Bit strings are represented as polynomials where each bit is a coefficient:

$$10011 \longleftrightarrow x^4 + x + 1 \quad (16)$$

$$1101 \longleftrightarrow x^3 + x^2 + 1 \quad (17)$$

5.4 GF(2) Arithmetic

All arithmetic is modulo-2 (XOR):

- Addition: $1 + 1 = 0$, $1 + 0 = 1$, $0 + 0 = 0$
- No carries or borrows
- Addition and subtraction are the **same operation** (XOR)

5.5 Key Principle

The set of valid codewords $T(x)$ are those polynomials that are **divisible by** $G(x)$:

$$T(x) \equiv 0 \pmod{G(x)} \quad (18)$$

5.6 CRC Encoding

Given message $M(x)$ and generator $G(x)$ of degree r :

Step 1: Multiply $M(x)$ by x^r (append r zero bits to message).

Step 2: Divide $x^r M(x)$ by $G(x)$ to get remainder $R(x)$:

$$x^r M(x) = Q(x) \cdot G(x) + R(x) \quad (19)$$

Step 3: Transmitted frame:

$$T(x) = x^r M(x) + R(x) \quad (20)$$

This is equivalent to $T(x) = x^r M(x) - R(x)$ (since $+$ $=$ $-$ in GF(2)).

Why this works:

$$T(x) = x^r M(x) + R(x) = Q(x) \cdot G(x) + R(x) + R(x) = Q(x) \cdot G(x) \quad (21)$$

So $T(x)$ is divisible by $G(x)$, making it a valid codeword.

5.7 CRC Decoding

At the receiver, divide the received frame $\hat{T}(x)$ by $G(x)$:

- If remainder = 0: **Accept** (no detected error)
- If remainder $\neq 0$: **Reject** (error detected)

Why this works: If $\hat{T}(x) = T(x) + E(x)$ where $E(x)$ is the error pattern:

$$\hat{T}(x) \bmod G(x) = \underbrace{T(x) \bmod G(x)}_{=0} + E(x) \bmod G(x) = E(x) \bmod G(x) \quad (22)$$

An error is **undetected** only if $E(x)$ is divisible by $G(x)$. A well-chosen $G(x)$ makes this extremely unlikely.

CRC Encoding Example

Let $M(x) = 1010$ (i.e., $x^3 + x$) and $G(x) = 101$ (i.e., $x^2 + 1$), so $r = 2$.

Step 1: $x^2 M(x) = 101000$ (append two zeros).

Step 2: Divide 101000 by 101 using modulo-2 long division:

```

      1 0 1 1
      -----
101 ) 101000
      101
      ---
        0000
         000
         ---
          0000
           000
           ---
            0000
             000
             ---
              010
               000
               ---
                10      <-- remainder R(x)
  
```

Wait – let me redo this division more carefully.

```

      1 0 0 0
      -----
101 ) 101000
      101
      ---
        0000
  
```

$101000 \div 101$: The leading 3 bits 101 match 101, so quotient bit is 1. XOR gives 000. Bring down next bit: 0000. Since leading bit is 0, quotient bit is 0. Continue: 0000, quotient 0. Finally 0000, quotient 0. Remainder is 00.

Actually, $101000 = 1000 \times 101 + R$. In GF(2): $(x^3)(x^2 + 1) = x^5 + x^3 = 101000$. So $R(x) = 0$.

Let us use a more illustrative example. Let $M = 1101$ and $G = 1011$, so $r = 3$.

Step 1: $x^3 M(x) = 1101000$ (append three zeros).

Step 2: Divide 1101000 by 1011:

```

      1 1 1 0
      -----
1011 ) 1101000
       1011
       ----
         1100
          1011
          ----
           1110
            1011
            ----
             1010
              1011
              ----
               001      <-- remainder R(x)
  
```

Step 3: $T = 1101000 + 001 = 1101001$.

Verification: Divide 1101001 by 1011:

6 CRC Encoding and Decoding Procedure (10 min)

6.1 Step-by-Step CRC Procedure

Transmitter:

1. Given: k -bit message M and $(r + 1)$ -bit generator G
2. Append r zeros to M : form $x^r M(x)$
3. Divide $x^r M(x)$ by $G(x)$ using modulo-2 long division
4. Transmit: $T = M$ followed by remainder R ($n = k + r$ bits total)

Receiver:

1. Receive $\hat{T}$ (possibly corrupted)
2. Divide $\hat{T}(x)$ by $G(x)$ using modulo-2 long division
3. If remainder = 0: accept frame
4. If remainder $\neq 0$: discard frame, request retransmission

Hands-On Activity: CRC Encoding

Problem: Given $M = 110101$ and $G = 1001$ ($r = 3$), compute the CRC and transmitted frame.

Try this yourself before looking at the solution!

Solution

Step 1: Append 3 zeros: $x^3M(x) = 110101000$.

Step 2: Divide 110101000 by 1001:

```

      1 1 1 1 0 0
      -----
1001 ) 110101000
      1001
      ----
        1001
        1001
        ----
          0001
          0000
          ----
            0010
            0000
            ----
              0101
              0000
              ----
                1010
                1001
                ----
                  0110
                  0000
                  ----
                    110      <-- remainder R(x)

```

Wait – let me be more careful with the alignment. The dividend has 9 bits and divisor has 4 bits, so the quotient has at most 6 bits.

```

      1 1 0 1 1 0
      -----
1001 ) 110101000
      1001
      ----
        1001
        1001
        ----
          0000
          0000
          ----
            0001
            0000
            ----
              0010
              0000
              ----
                0100
                0000
                ----
                  100      <-- R not right, let me retry

```

Let me step through bit-by-bit. $M' = 110101000$:

Position 1–4: 1101. Since MSB is 1, XOR with 1001: $1101 \oplus 1001 = 0100$. Bring down next bit (0): 01000.

Position: 0100. MSB is 0, so quotient bit is 0. Bring down next bit (1): 01001.

Position: 1001. MSB is 1, XOR with 1001: $1001 \oplus 1001 = 0000$. Bring down next bit (0):

7 CRC Performance and Standard Polynomials (10 min)

7.1 CRC Error Detection Capabilities

A well-chosen generator polynomial $G(x)$ can detect:

Error Type	Condition
All single-bit errors	If $G(x)$ has more than one term
All double-bit errors	If $G(x)$ has a factor not dividing $x^j + 1$ for any $j \leq n$
All odd numbers of errors	If $G(x)$ has $(x + 1)$ as a factor
All burst errors of length $\leq r$	Always (for any $G(x)$ of degree r)
Burst errors of length $r + 1$	Detected with probability $1 - 2^{-(r-1)}$
Burst errors of length $> r + 1$	Detected with probability $1 - 2^{-r}$

Table 3: CRC error detection capabilities

7.2 Burst Error Definition

A **burst error** of length l is a contiguous sequence of l bits where the first and last bits are in error (intermediate bits may or may not be in error).

Burst Error Illustration

Consider a received frame where errors are marked with $\times$:

0 0 $\times$ 1 0 $\times$ 0 0 0

This is a burst error of length 4 (from bit position 3 to bit position 6). The bits between the first and last error may or may not be corrupted — the burst length is determined by the span from first to last error bit.

7.3 Why Single-Bit Errors Are Always Detected

A single-bit error at position i gives $E(x) = x^i$. For this to go undetected, $G(x)$ must divide x^i . But if $G(x)$ has more than one term (i.e., at least two non-zero coefficients), it cannot divide x^i . Therefore, all single-bit errors are detected.

7.4 Why Burst Errors $\leq r$ Are Always Detected

A burst error of length $\leq r$ can be written as $E(x) = x^j B(x)$ where $B(x)$ has degree $< r$. Since $G(x)$ has degree r , it cannot divide $B(x)$ (degree too small). And $G(x)$ does not divide x^j (it has a constant term). Therefore, $G(x)$ cannot divide $E(x)$.

7.5 Standard CRC Polynomials

7.6 CRC-16 Performance

CRC-16 ($x^{16} + x^{15} + x^2 + 1$) detects:

Name	Polynomial	r	Common Use
CRC-8	$x^8 + x^2 + x + 1$	8	ATM header
CRC-16	$x^{16} + x^{15} + x^2 + 1$	16	USB, Modbus
CRC-CCITT	$x^{16} + x^{12} + x^5 + 1$	16	HDLC, Bluetooth
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22}$ $+ x^{16} + x^{12} + x^{11} + x^{10}$ $+ x^8 + x^7 + x^5 + x^4$ $+ x^2 + x + 1$	32	Ethernet, WiFi, ZIP

Table 4: Standard CRC polynomials used in networking

- All single, double, and triple bit errors
- All odd numbers of errors (since $(x + 1)$ is a factor)
- All burst errors of length ≤ 16
- Most longer burst errors (probability of missing: $\approx 2^{-16} \approx 1.5 \times 10^{-5}$)

These guarantees hold as long as the message is shorter than 2^{15} bits (≈ 32 KB).

7.7 CRC-32 Performance

CRC-32 is used in Ethernet (802.3) and WiFi (802.11) and provides extremely strong error detection:

- Detects all burst errors of length ≤ 32
- For random errors: probability of undetected error $\approx 2^{-32} \approx 2.3 \times 10^{-10}$
- Practical guarantee: detects virtually all errors in frames up to 12,000 bits
- Implemented in hardware using **shift registers with XOR gates**, making it extremely fast (computed at wire speed)

7.8 CRC in Practice

- CRC is computed by the **network interface card (NIC)** in hardware
- The 4-byte CRC-32 is appended as the **Frame Check Sequence (FCS)** in Ethernet and WiFi frames
- Receiver computes CRC on received frame; if non-zero remainder, the frame is **silently discarded**
- No error correction — higher layers (e.g., TCP) handle retransmission

8 Summary (5 min)

8.1 Lecture 6 Key Takeaways

1. **Single-bit error correction** requires $n + 1 \leq 2^{n-k}$ check bits (necessary condition)
2. **Error detection is more efficient** than correction for typical networking scenarios (approximately 5× throughput advantage)
3. **Repetition codes** offer better P_e but at very high throughput cost
4. **CRC codes** treat bit strings as polynomials over $\text{GF}(2)$
5. **CRC encoding**: append remainder of $x^r M(x) \div G(x)$
6. **CRC decoding**: divide received frame by $G(x)$; remainder = 0 means valid
7. **Standard CRCs** (CRC-16, CRC-32) detect virtually all common error patterns
8. CRC-32 is used in **Ethernet and WiFi** — the most common link-layer protocols

8.2 Connection to Networking

Error control coding is fundamental to reliable data delivery:

- **Ethernet (802.3)** uses CRC-32 for frame check sequence (FCS)
- **WiFi (802.11)** uses CRC-32 for FCS
- **TCP/UDP** use simpler Internet checksums (ones' complement sum)
- **IP** uses header checksum for integrity
- Different layers may use different error control strategies

8.3 Looking Ahead

Next topics in the Data Link Layer:

- Framing and frame formats
- Medium access control (MAC) protocols
- Ethernet and WiFi protocols in detail

References

- [1] Kurose, J.F. and Ross, K.W. *Computer Networking: A Top-Down Approach*, 8th Edition. Pearson, 2021.
- [2] Forouzan, B.A. *Data Communications and Networking*, 5th Edition. McGraw-Hill, 2013.
- [3] Wikipedia contributors. "Cyclic redundancy check." *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Cyclic_redundancy_check

- [4] Wikipedia contributors. “Mathematics of cyclic redundancy checks.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Mathematics_of_cyclic_redundancy_checks
- [5] Wikipedia contributors. “Computation of cyclic redundancy checks.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Computation_of_cyclic_redundancy_checks
- [6] Wikipedia contributors. “Hamming distance.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Hamming_distance
- [7] Wikipedia contributors. “Block code.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Block_code
- [8] Wikipedia contributors. “Error detection and correction.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Error_detection_and_correction

Additional Resources

Textbooks:

- Peterson, W.W. and Weldon, E.J. *Error-Correcting Codes*, 2nd Edition. MIT Press, 1972.
- Lin, S. and Costello, D.J. *Error Control Coding*, 2nd Edition. Pearson, 2004.

Key Formulas:

Formula	Description
$e_c + e_d \leq d_{\min} - 1$	$d_{\min}$ theorem
$n + 1 \leq 2^{n-k}$	Single-bit EC necessary condition
$T(x) = x^r M(x) + R(x)$	CRC encoding
$x^r M(x) = Q(x)G(x) + R(x)$	CRC division
$\hat{T}(x) \bmod G(x) = 0 \implies \text{accept}$	CRC decoding

Table 5: Key formulas from Lecture 6

Tools:

- Interactive CRC calculator: exploration notebook `lecture_05_06_exploration.py`
- Online CRC calculators for verification